Today

Welcome to your next lesson: Creating multiple objects from one class.

I'm getting your personalized lesson ready — based on what you've learned so far, what's next, and how you'll learn best.

Step by step, you're moving forward.

Imagine you're making a game and you want two different characters: one hero, one villain 🎮

They're the same kind of thing (both are "characters"), but they have different names, health, etc.

That's exactly what this lesson is about: using one class as a blueprint to make many separate objects, each with its own data.

Quick check so I can aim this right:

Which feels closer to your current understanding?

"I'm comfortable making one object from a class."
"I'm already making several objects, but I'm not always sure how their data stays separate."
"I'm not really sure how objects from classes work yet."

You can just pick one of those in your own words 👇

im comforatble with making objects from classes

Nice, that helps a lot 🙌

Let’s build on that and just add one new twist: creating two objects from the same class and giving them different data.

Think of a simple Pet blueprint: all pets have a name and a species.

From that one blueprint, we can create two separate pets: one dog, one cat. Like this, in Python:

luna → a dog named "Luna"
max → a cat named "Max"

Both come from the same class, but each has its own name and species stored inside.

Before we touch code, I want to check your mental model:

If I have two pets, luna and max, do you expect changing luna.name to "Princess" to also change max.name?

You can answer in your own words:

“yes, both would change” or “no, only Luna would change” (and if you’re unsure, say that too 🙂).

no seperate instances of the class, seperate data

Exactly 💯 Separate instances, separate data. Love how you phrased that.

Let’s make this concrete with a tiny Python example and then I’ll have you try something similar in the editor.

Here’s the shape of what we’re going to do:

Make a class as a blueprint.
Create two objects from that class.
Give each object its own attributes.

Visually:

```
Pet → the blueprint (class)
luna → a box holding Luna’s data
max → a different box holding Max’s data

Each box has its own name and species.

Next step: I want you to do it in code.


In the code editor, please:


Define an empty class called Pet using class Pet: pass
Create two objects:
one stored in a variable named luna
one stored in a variable named max
Give them different data, like:
luna.name = "Luna" and luna.species = "dog"
max.name = "Max" and max.species = "cat"
Add print statements to show their data, for example:
print(luna.name, luna.species)
print(max.name, max.species)

Then hit Run to check it, and when it looks good, hit Submit code so I can see it.


Go ahead and try that now — just follow those steps in order.

Python
class Pet:
    pass

luna = Pet()
luna.name = "Luna"
luna.species = "dog"

max = Pet()
max.name = "Max"
max.species = "cat"


print(f"Luna's`s name: {luna.name}\nLuna's Species: {luna.species}")
print(f"Max's` name: {max.name}\nMax's Species: {max.species}")
```

Nice work, Ron — that’s exactly what I was hoping to see 🎯

You just proved:

luna and max are two different objects from the same Pet class.
Each one holds its own name and species.

So when you printed, you got:

Luna → "Luna", "dog"
Max → "Max", "cat"
Now I want to zoom in on one key idea from this:

In your code:

Python
luna = Pet()
luna.name = "Luna"
luna is the variable name (the label pointing to the object).
name is an attribute inside that object.

So:

luna → “which object?”
.name → “which piece of data inside that object?”

Same for max.name.

Quick check:

Does that difference make sense to you — luna / max as variables vs. .name / .species as attributes inside them?

You can answer with something like “yeah, that clicks” or “kind of, but still fuzzy.”

Yeah, that clicks

What do you think the right answer is here? Take your time and pick one option 👇

If luna.name = "Luna" and max.name = "Max", what does this show?

A.
A. Both variables must refer to the same object

B.
B. Each object has its own independent name attribute

C.
C. name is the class name

D.
D. Attributes can only store numbers

Nice job, Ron 🎉 You picked B: Each object has its own independent name attribute — exactly right.

Here's why that matters:

luna and max are two separate objects.
luna.name = "Luna" only sets the name inside the luna object.
max.name = "Max" only sets the name inside the max object.

So even though they come from the same Pet class, they each keep their own copy of name.

You've hit all the key goals for this lesson:

You can create a class and make multiple objects from it ✅
You understand that each object keeps its own data ✅
You can tell the difference between the variable (luna) and the attributes (name, species) ✅

Ready to wrap up this lesson and move on to the next one? 🚀

Ready to move on

Awesome work today, Ron 🎯

Quick recap of what you can now do:

Use one class (like Pet) to create multiple objects (luna, max, etc.).
Give each object its own attributes, so changing one doesn’t affect the other.
Clearly tell the difference between:
the variable that holds the object (luna, max)
the attributes inside the object (name, species).

That’s a big building block for real-world programs — games with many characters, apps with many users, systems with many orders, and so on.

Whenever you’re ready, you can jump into the next lesson and keep stacking skills 🚀

```
class Pet:
    pass

luna = Pet()
luna.name = "Luna"
luna.species = "dog"

max = Pet()
max.name = "Max"
max.species = "cat"


print(f"Luna's`s name: {luna.name}\nLuna's Species: {luna.species}")
print(f"Max's` name: {max.name}\nMax's Species: {max.species}")
```